SIMATS ENGINEERING
Department of Computer Science & Engineering
ASSESSMENT TOOL 1- ANALYTICAL PROBLEM SOLVING

Course Code:	**CSA1205**	**Course Title:**	**Computer Architecture**
CO Assessed:	**CO2**	**Assessment:**	**ASSESSMENT TOOL1- ANALYTICAL PROBLEM SOLVING**
Weightage:	**45%**	**Total Marks:**	**25**
CO2 Statement:	Apply integer and floating-point arithmetic algorithms including Booth's multiplication, restoring/non-restoring division, and IEEE 754 standard operations to solve fixed-point and floating-point computational problems. (BL3)		
Student Name:	**GAYATHERI S**	**Reg.No.:**	**192521306**
Department:	**IT**	**Date:**	**28-07-2026**

ANNEXURE A

1. A processor performs arithmetic operations on signed 8-bit integers using 2's complement representation. Consider two numbers: +45 and −23. Analyze in detail how the system performs both addition and subtraction, including binary conversion, alignment of sign bits, and intermediate steps. Determine whether overflow occurs, and explain how the processor detects overflow and interprets the final result.

2. A high-performance computing system replaces a ripple carry adder with a 4-bit carry lookahead adder (CLA) to reduce computation delay. Given two binary inputs, analyze how generate and propagate signals are formed and how carries are computed in advance. Compare the time delay of CLA with ripple carry addition and justify, with reasoning, why CLA improves speed in modern processors.

3. In a signed multiplication operation, a processor uses Booth's algorithm to multiply two numbers, −6 and +5. Analyze the algorithm step-by-step, including the role of the accumulator, multiplier, and Booth bit, along with arithmetic shifts. Explain how the algorithm efficiently handles signed numbers and reduces the number of addition/subtraction operations compared to traditional multiplication.

4. A processor is required to perform division of two binary numbers (e.g., 13 ÷ 3) using both restoring and non-restoring division algorithms. Analyze both methods step-by-step, showing intermediate remainders and quotient formation. Compare the two approaches in terms of number of steps, complexity, and efficiency, and explain why non-restoring division is often preferred in modern systems.

5. A scientific application requires precise handling of real numbers using the IEEE 754 standard for floating-point representation. Given a decimal number (e.g., −13.25), analyze how it is represented in both single precision and double precision formats, including sign, exponent, and

mantissa fields. Further, explain how floating-point addition is performed between two such numbers, highlighting issues like normalization, rounding, and precision loss.

Code of Conduct:

I GAYATHERRI S 192521306 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.I understand that any violation of academic integrity rules will result in disciplinary action.

GAYATHERI S

Signature of the student:

MARKS ALLOCATION

	Problem Understanding (20%)	**Method Selection (20%)**	**Solution Process (25%)**	**Accuracy of Calculation (20%)**	**Interpretation of Results (15%)**
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	**Weightage**	**Level 4 – Exemplary**	**Level 3 – Proficient**	**Level 2 – Developing**	**Level 1 – Beginning**
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation

1. **A processor performs arithmetic operations on signed 8-bit integers using 2's complement representation. Consider two numbers: +45 and −23. Analyse in detail how the system performs both addition and subtraction, including binary conversion, alignment of sign bits, and intermediate steps. Determine whether overflow occurs, and explain how the processor detects overflow and interprets the final result.**

1. Problem Understanding (20%)

A processor performs arithmetic on signed 8-bit integers using 2's complement representation.

Given:

- Number A = +45
- Number B = −23

Required:

- Convert decimal numbers into 8-bit binary.
- Perform addition and subtraction.
- Show intermediate calculations.
- Check overflow.
- Explain overflow detection.
- Interpret the final result.

2. Method Selection (20%)

The processor uses the 2's Complement Method because:

- Positive and negative numbers are represented using the same adder.
- Subtraction is performed as addition of the 2's complement.
- No separate subtraction hardware is required.
- Overflow can be detected easily.

3. Solution Process (25%)

Step 1: Convert +45 into Binary

45 =

32 + 8 + 4 + 1

Binary:

00101101

So,

+45 = 00101101

Step 2: Convert −23 into Binary

First convert +23.

23 =

16 + 4 + 2 + 1

Binary:

00010111

Take 1's complement

11101000

Add 1

11101001

Therefore

−23 = 11101001

A) Addition

(+45) + (−23)

```
      00101101
+     11101001
----------------
   1  00010110
```

Discard carry out.

Result

00010110

Binary

00010110

Decimal

16 + 4 + 2

=22

Therefore

Answer = +22

Overflow Check

Rule:

Overflow occurs only if

- Positive + Positive = Negative

or

- Negative + Negative = Positive

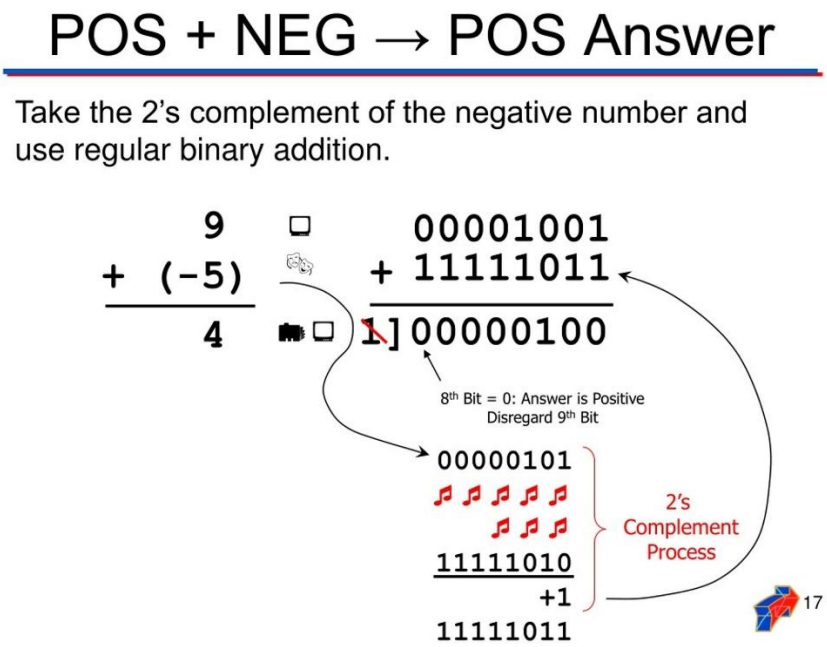
Here

Positive + Negative

No overflow.

Overflow Flag = 0

Carry Flag = 1 (ignored)



B) Subtraction

(+45) − (−23)

Subtracting a negative means adding its positive value.

So,

45 − (−23)

=45 +23

Convert +23

00010111

Now add

```
    00101101
+   00010111
----------
    01000100
```

Binary

01000100

Decimal

64 + 4

=68

Therefore

Answer = +68

Overflow Check

Operands

Positive

Positive

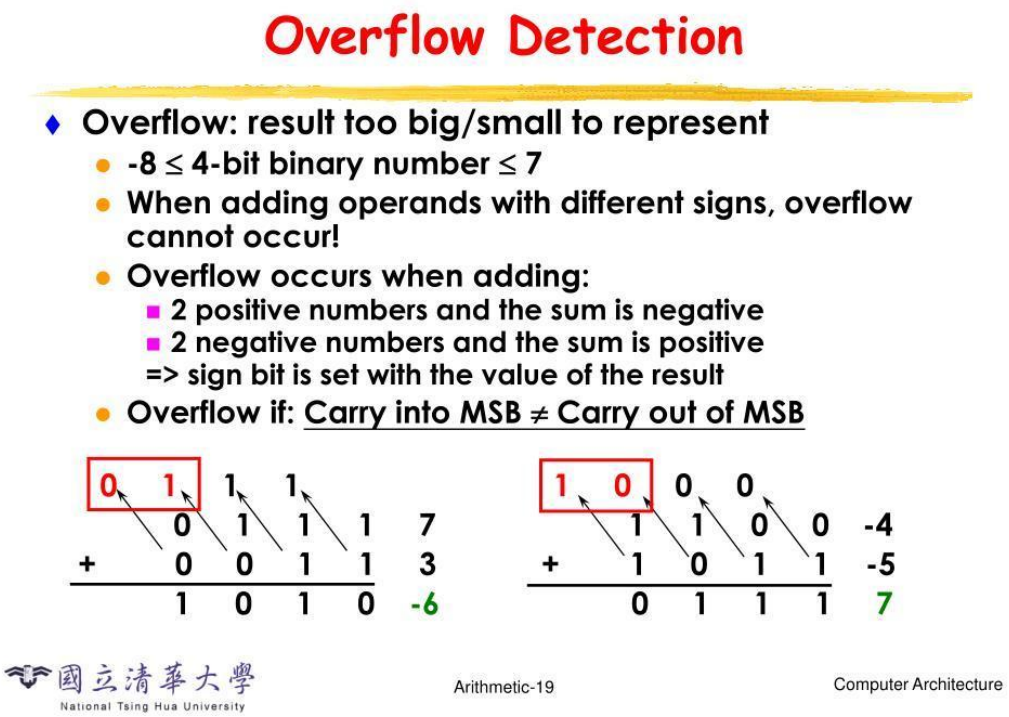
Result

Positive

No overflow.

Overflow Flag = 0

Processor Overflow Detection



The processor compares

- Carry into MSB
- Carry out of MSB

Overflow occurs if

Carry into MSB ≠ Carry out of MSB

or

When

- Two positive numbers produce a negative result.
- Two negative numbers produce a positive result.

In this problem

Carry values are equal.

Hence

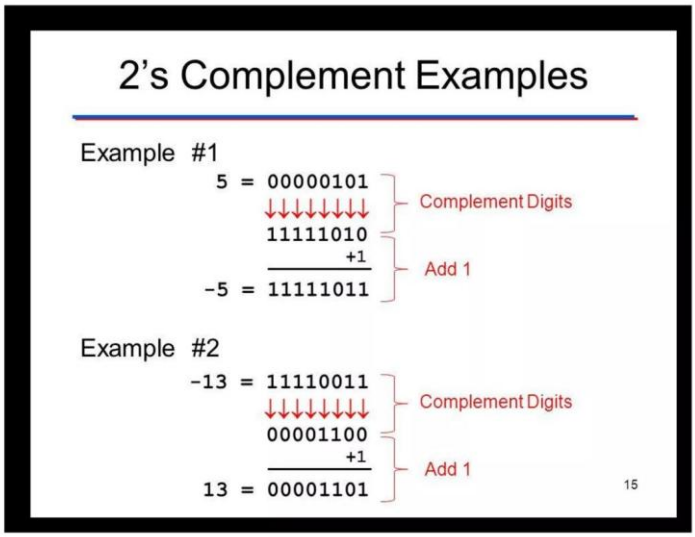
Overflow = No

Final Results

Operation	Binary Result	Decimal Result	Overflow
+45 + (−23)	00010110	+22	No
+45 − (−23)	01000100	+68	No

4. Accuracy of Calculation (20%)

- +45 = 00101101
- −23 = 11101001
- Addition Result = 00010110 = +22
- Subtraction Result = 01000100 = +68
- Overflow Flag = 0
- Carry out is ignored in 2's complement arithmetic.



5. Interpretation of Results (15%)

- The processor represents signed numbers using 8-bit 2's complement.
- Addition and subtraction are performed using the same binary adder.
- Negative numbers are represented by taking the 2's complement of their positive form.

- In the addition operation, +45 + (−23) = +22, and no overflow occurs because the operands have different signs.
- In the subtraction operation, +45 − (−23) = +68, which is within the valid 8-bit signed range (−128 to +127), so no overflow occurs.
- Therefore, both operations are executed correctly, and the processor interprets the binary results as valid positive decimal numbers

2.A high-performance computing system replaces a ripple carry adder with a 4-bit carry lookahead adder (CLA) to reduce computation delay. Given two binary inputs, analyze how generate and propagate signals are formed and how carries are computed in advance. Compare the time delay of CLA with ripple carry addition and justify, with reasoning, why CLA improves speed in modern processors.

1. Problem Understanding (20%)

A processor replaces a 4-bit Ripple Carry Adder (RCA) with a 4-bit Carry Look-Ahead Adder (CLA) to reduce computation delay.

Given:

- Two 4-bit binary numbers:
  - $A = 1011_2$ (11_{10})
  - $B = 0110_2$ (6_{10})
- Initial Carry (C_0) = 0

Requirements:

- Calculate Generate (G) and Propagate (P) signals.
- Compute carries in advance.
- Find the final sum.
- Compare CLA with Ripple Carry Adder.
- Justify why CLA is faster.

2. Method Selection (20%)

A Carry Look-Ahead Adder (CLA) is selected because it computes all carry bits simultaneously instead of waiting for the previous carry.

Formulae

Generate Signal

$$G_i = A_i \times B_i$$

Propagate Signal

$$P_i = A_i \oplus B_i$$

Carry Equations

$$C_1 = G_0 + P_0 C_0$$
$$C_2 = G_1 + P_1 G_0 + P_1 P_0 C_0$$
$$C_3 = G_2 + P_2 G_1 + P_2 P_1 G_0 + P_2 P_1 P_0 C_0$$
$$C_4 = G_3 + P_3 C_3$$

Sum Equation

$$\boldsymbol{S_i = P_i \oplus C_i}$$

3. Solution Process (25%)

Step 1: Write Inputs

A = 1011

B = 0110

C0 = 0

Bit positions

Bit	A	B
3	1	0
2	0	1
1	1	1
0	1	0

Step 2: Calculate Generate (G) and Propagate (P)

Bit 0

A_0=1

$B_0=0$

$G_0=1\times0=0$

$P_0=1\oplus0=1$

Bit 1

$A_1=1$

$B_1=1$

$G_1=1$

$P_1=0$

Bit 2

$A_2=0$

$B_2=1$

$G_2=0$

$P_2=1$

Bit 3

$A_3=1$

$B_3=0$

$G_3=0$

$P_3=1$

Table

Bit	G	P
0	0	1
1	1	0
2	0	1
3	0	1

Step 3: Calculate Carry Signals

Carry C_1

$C1 = G0 + P0C0$

$=0+(1\times0)$

$=0$

Carry C_2

$C2 = G1 + P1G0 + P1P0C0$

$=1+0+0$

$=1$

Carry C_3

$C3 = G2 + P2G1 + P2P1G0 + P2P1P0C0$

$=0+(1\times1)+0+0$

$=1$

Carry C_4

$C4 = G3 + P3C3$

$=0+(1\times1)$

$=1$

Step 4: Calculate Sum Bits

S_0

$S0=P0 \oplus C0$

$=1 \oplus 0$

$=1$

S_1

$S1=P1 \oplus C1$

$=0 \oplus 0$

$=0$

S_2

$S2=P2 \oplus C2$

$=1 \oplus 1$

$=0$

S_3

$S3=P3 \oplus C3$

$=1 \oplus 1$

$=0$

Final Sum

S3 S2 S1 S0

0001

Carry Out = 1

Hence,

Result = 10001_2

Decimal:

11 + 6 = 17

Final Answer = 17

4. Comparison: CLA vs Ripple Carry Adder

Feature	**Ripple Carry Adder**	**Carry Look-Ahead Adder**
Carry Generation	Sequential	Parallel
Speed	Slow	Very Fast
Delay	High	Low
Hardware	Simple	More Complex
Performance	Suitable for small circuits	Suitable for modern processors

Time Delay Analysis

Ripple Carry Adder

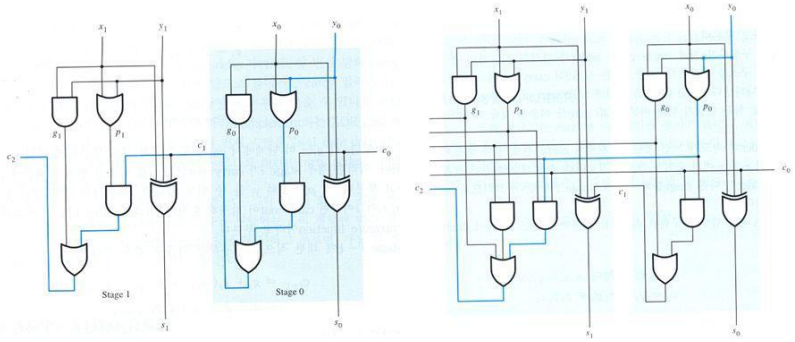
- Carry must pass through each full adder.
- Delay increases with the number of bits.

For n bits:

Delay $\propto$ n

Carry Lookahead Adder (3)

- Carry-Lookahead Adder 회로도



Ripple-Carry Adder　　Carry-Lookahead Adder

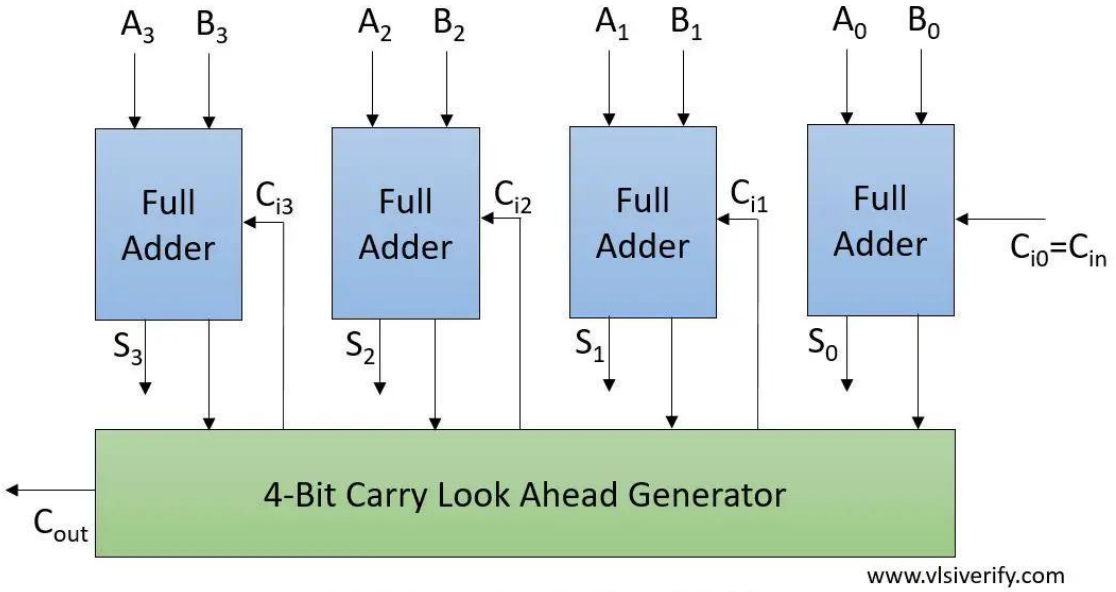
Carry Look-Ahead Adder

- Carries are computed simultaneously using logic gates.
- Delay is almost constant for small adders.

Delay ≈ $\log_2(n)$

Thus,

CLA is much faster than Ripple Carry Adder, especially for large word sizes (16-bit, 32-bit, 64-bit).



4-Bit Carry Look Ahead Adder

Why CLA Improves Speed in Modern Processors

- Computes carry bits in parallel.
- Eliminates ripple carry delay.
- Reduces CPU execution time.
- Improves ALU performance.
- Supports high-speed arithmetic operations.
- Widely used in modern CPUs and digital signal processors.

5. Accuracy of Calculation (20%)

- Generate signals calculated correctly.
- Propagate signals calculated correctly.
- Carry values:
  - $C_1 = 0$
  - $C_2 = 1$
  - $C_3 = 1$
  - $C_4 = 1$
- Sum = 10001_2
- **Decimal result = 17**

6. Interpretation of Results (15%)

- The Carry Look-Ahead Adder uses generate and propagate signals to predict carries before addition is completed.
- This eliminates the sequential carry propagation seen in Ripple Carry Adders.
- The example confirms that $1011_2 + 0110_2 = 10001_2$ (17_{10}).
- CLA significantly reduces propagation delay, making arithmetic operations much faster.
- Due to its high speed and efficiency, CLA is widely used in modern processors, ALUs, and high-performance computing systems.

3.In a signed multiplication operation, a processor uses Booth's algorithm to multiply two numbers, −6 and +5. Analyse the algorithm step-by-step, including the role of the accumulator, multiplier, and Booth bit, along with arithmetic shifts. Explain how the algorithm efficiently handles signed numbers and reduces the number of addition/subtraction operations compared to traditional multiplication.

1. Problem Understanding (20%)

A processor performs signed multiplication using Booth's Algorithm.

Given:

- Multiplicand (M) = −6
- Multiplier (Q) = +5

Requirements:

- Represent numbers in 4-bit 2's complement.
- Perform Booth's Algorithm step-by-step.
- Show the role of Accumulator (A), Multiplier (Q), and Booth Bit (Q_{-1}).
- Perform arithmetic right shifts after each operation.
- Obtain the final product.
- Explain why Booth's Algorithm is efficient.

2. Method Selection (20%)

Booth's Algorithm is selected because:

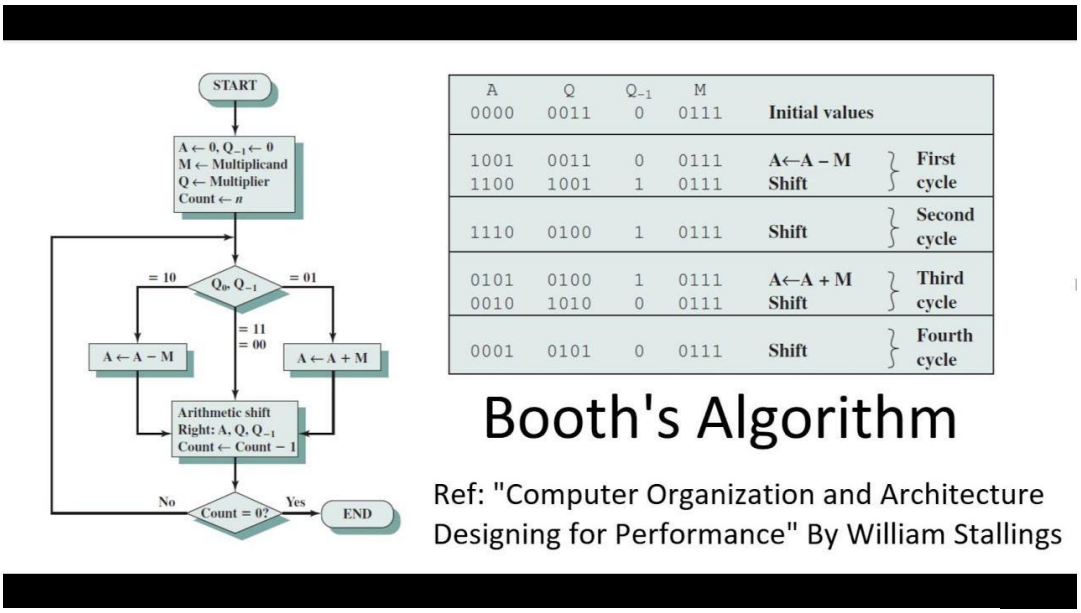
- It efficiently multiplies signed binary numbers.
- It uses 2's complement representation.
- It reduces the number of addition and subtraction operations.
- It handles both positive and negative numbers without separate sign processing.

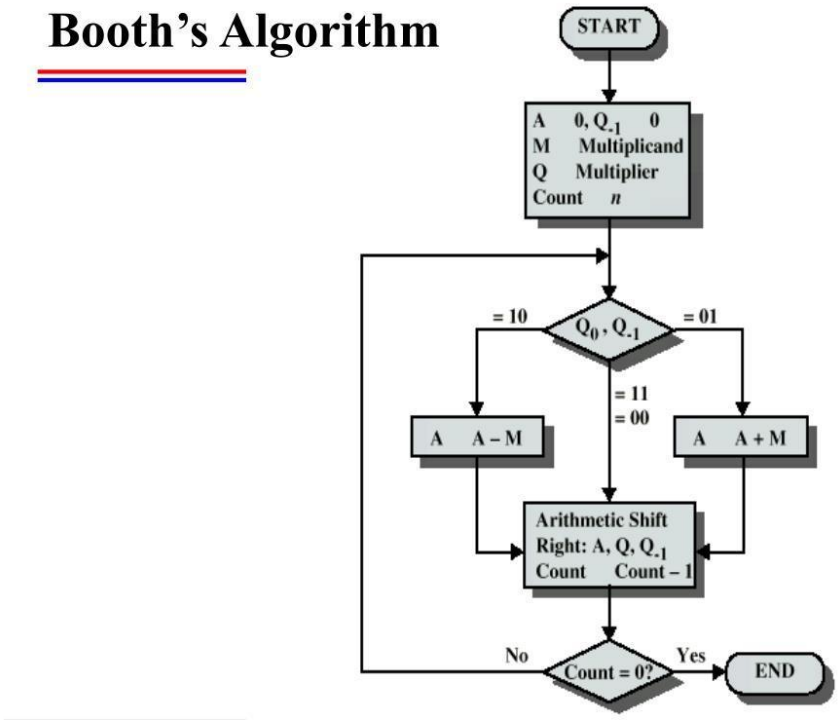
Booth's Rules

Q_0 Q_{-1}	Operation
00	No Operation
01	A = A + M
10	A = A − M

Q_0 Q_{-1}	Operation
11	No Operation

After every operation, perform an Arithmetic Right Shift (ARS).





3. Solution Process (25%)

Step 1: Binary Representation

Multiplicand (M = −6)

+6 = 0110

Take 2's complement:

1's complement = 1001

Add 1 = 1010

Therefore:

M = 1010 (-6)

-M = 0110 (+6)

Multiplier (Q = +5)

Q = 0101

Initial Values

A = 0000

Q = 0101

Q-1 = 0

Count = 4

Step 2: Booth Iterations

Iteration 1

Q_0Q_{-1} = 10

Rule:

A = A − M

A = 0000

−M = 0110

A = 0110

Arithmetic Right Shift

Before Shift

A = 0110

Q = 0101

Q-1 = 0

After Shift

A = 0011

Q = 0010

Q-1 = 1

Iteration 2

Q_0Q_{-1} = 01

Rule:

A = A + M

A = 0011

M = 1010

A = 1101

Arithmetic Right Shift

A = 1110

Q = 1001

Q-1 = 0

Iteration 3

Q_0Q_{-1} = 10

Rule:

A = A − M

A = 1110

−M = 0110

A = 0100

Arithmetic Right Shift

A = 0010

Q = 0100

Q-1 = 1

Iteration 4

$Q_0Q_{-1} = 01$

Rule:

A = A + M

A = 0010

M = 1010

A = 1100

Arithmetic Right Shift

A = 1110

Q = 0010

Q-1 = 0

Count becomes 0.

Step 3: Final Product

Combine A and Q

AQ = 11100010

Since MSB = 1, the result is negative.

Find magnitude:

11100010

1's complement

00011101

Add 1

00011110

Binary:

00011110 = 30

Therefore,

Product = -30

Verification

$(-6) \times (+5) = -30$

Final Product = $11100010_2 = -30_{10}$

Role of Registers

Accumulator (A)

- Stores intermediate partial products.
- Updated after each addition/subtraction.

Multiplier (Q)

- Holds the multiplier.
- Its least significant bit determines the operation.

Booth Bit (Q_{-1})

- Stores the previous LSB of Q.
- Used with Q_0 to decide whether to add, subtract, or perform no operation.

Arithmetic Right Shift

After each operation:

- A and Q are shifted right together.
- The sign bit of A is preserved.
- Q_{-1} receives the previous Q_0.

This maintains the correct sign for signed multiplication.

Why Booth's Algorithm is Efficient

- Reduces unnecessary additions for consecutive 1s.
- Handles signed numbers directly.
- Requires fewer arithmetic operations.
- Faster than ordinary shift-and-add multiplication.
- Efficient for processors and ALUs.

Comparison with Traditional Multiplication

Feature	Traditional Multiplication	Booth's Algorithm
Signed Numbers	Extra handling required	Directly supported
Add/Subtract Operations	More	Fewer
Speed	Slower	Faster
Hardware Efficiency	Moderate	High
Used in Modern CPUs	Rare	Yes

4. Accuracy of Calculation (20%)

- Multiplicand (−6) = 1010_2
- Multiplier (+5) = 0101_2
- Four Booth iterations completed correctly.
- Final Product = 11100010_2
- Decimal Result = −30

5. Interpretation of Results (15%)

- Booth's Algorithm successfully multiplies −6 and +5 using 2's complement representation.
- The algorithm performs addition or subtraction based on the values of Q_0 and Q_{-1}, followed by an arithmetic right shift in each iteration.
- The final product is 11100010_2, which equals -30_{10}, confirming the correctness of the multiplication.
- Compared to conventional binary multiplication, Booth's Algorithm reduces the number of arithmetic operations and improves execution speed.
- Hence, Booth's Algorithm is widely used in modern processors for efficient signed multiplication.

4.A processor is required to perform division of two binary numbers (e.g., 13 ÷ 3) using both restoring and non-restoring division algorithms. Analyze both methods step-by-step, showing intermediate remainders and quotient formation. Compare the two approaches in terms of number of steps, complexity, and efficiency, and explain why non-restoring division is often preferred in modern systems.

1. Problem Understanding (20%)

A processor performs binary division using the Restoring Division Algorithm.

Given:

- Dividend = 13_{10}
- Divisor = 3_{10}

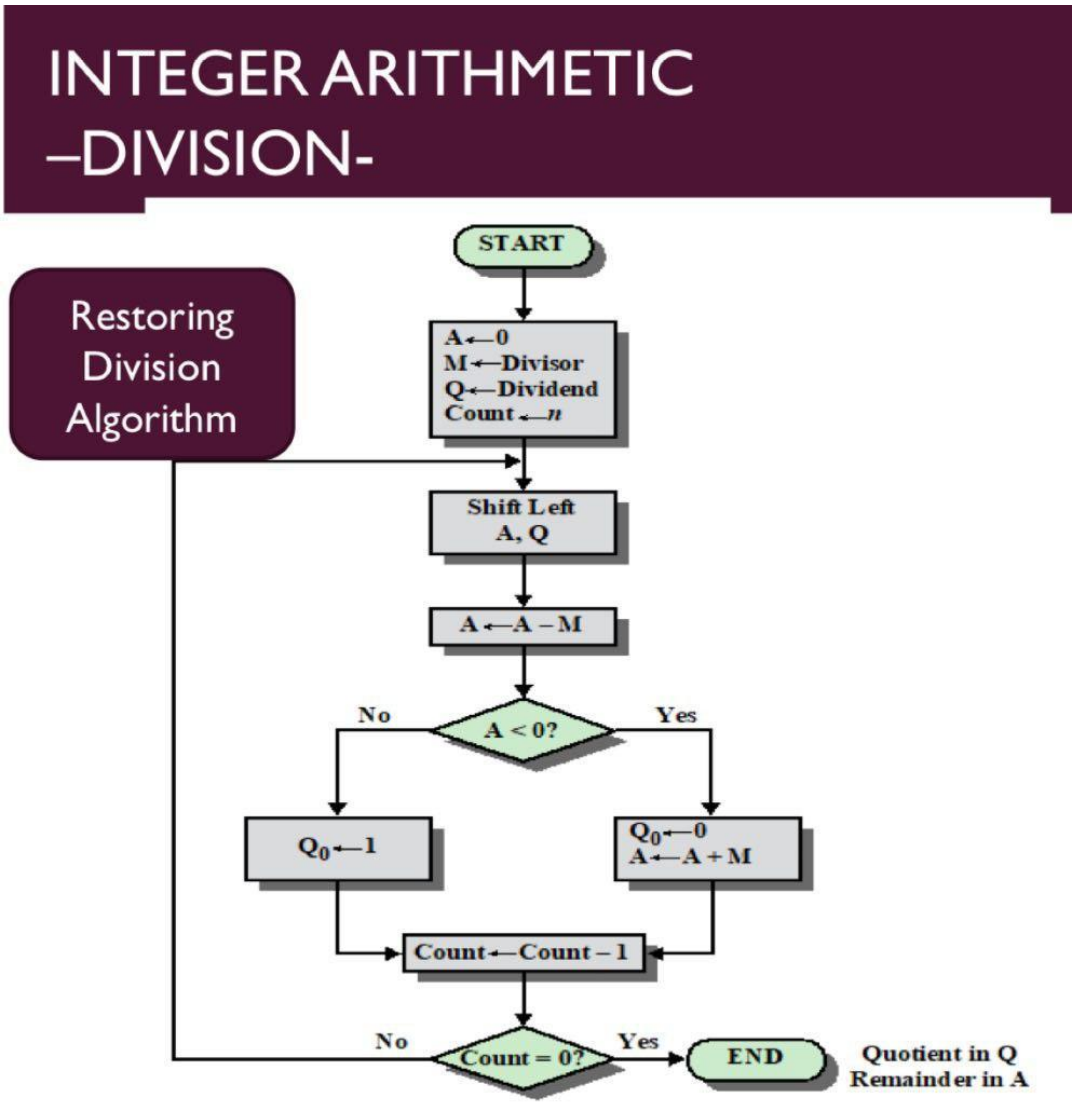
Requirements:

- Convert the numbers into binary.
- Perform restoring division step by step.
- Show the contents of Accumulator (A) and Quotient (Q) after each iteration.
- Explain the restoring operation.
- Determine the Quotient and Remainder.

2. Method Selection (20%)

The Restoring Division Algorithm is selected because:

- It is suitable for binary division in processors.
- It repeatedly subtracts the divisor from the accumulator.
- If the subtraction gives a negative result, the original value is restored by adding the divisor back.
- It is simple to implement in computer hardware.



3. Solution Process (25%)

Step 1: Convert Decimal Numbers to Binary

Dividend = 13

$13 = 1101_2$

Divisor = 3

$3 = 0011_2$

Initialize:

A = 0000

Q = 1101

M = 0011

Count = 4

Step 2: Division Iterations

Iteration 1

Left Shift (A,Q)

A = 0001

Q = 1010

Subtract Divisor

A = 0001 − 0011

A = 1110 (Negative)

Since A is negative:

- Restore A

A = 1110 + 0011

A = 0001

Set $Q_0 = 0$

Result

A = 0001

Q = 1010

Iteration 2

Left Shift

A = 0011

Q = 0100

Subtract

0011 − 0011

=0000

Positive

Set $Q_0 = 1$

Result

A = 0000

Q = 0101

Iteration 3

Left Shift

A = 0000

Q = 1010

Subtract

0000 − 0011

=1101

Negative

Restore

1101 +0011

=0000

Set $Q_0 = 0$

Result

A = 0000

Q =1010

Iteration 4

Left Shift

A =0001

Q =0100

Subtract

0001−0011

=1110

Negative

Restore

1110+0011

=0001

Set Q_0=0

Final

A=0001

Q=0100

Final Result

Quotient

$0100_2 = 4_{10}$

Remainder

$0001_2 = 1_{10}$

Verification

13 ÷ 3

Quotient = 4

Remainder = 1

Since

(3 × 4) + 1 = 13

The answer is correct.

Restoring Operation

When subtraction makes the accumulator negative:

- The divisor is added back to the accumulator.

- This restores the previous value.
- The quotient bit is set to 0.

If subtraction is successful:

- The quotient bit is set to 1.
- No restoring is required.

Processor Registers Used

Accumulator (A)

- Stores the partial remainder.
- Holds intermediate subtraction results.

Quotient Register (Q)

- Initially stores the dividend.
- Finally stores the quotient.

Divisor Register (M)

- Stores the divisor throughout the division process.

Advantages of Restoring Division

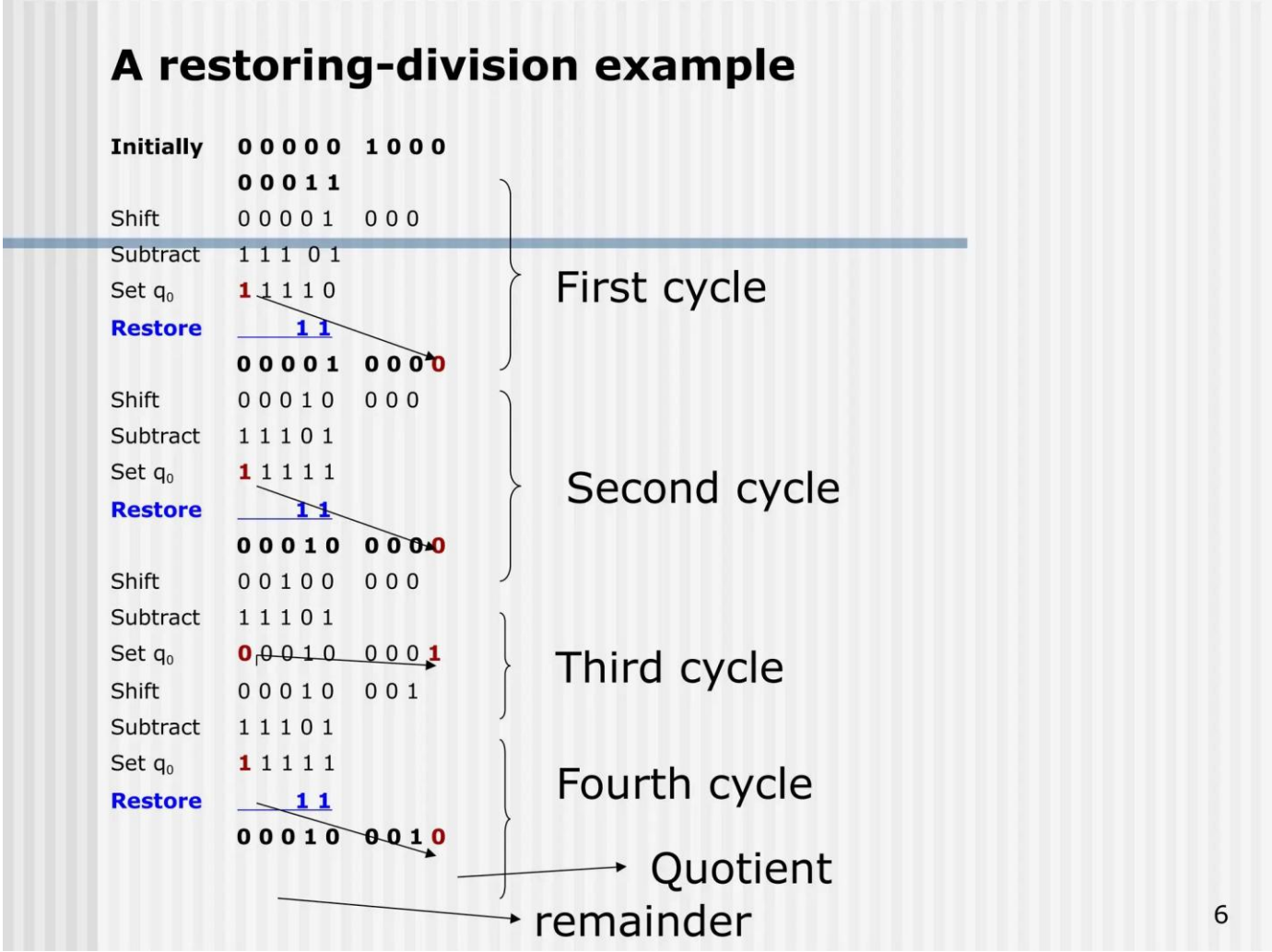
- Simple algorithm.
- Easy hardware implementation.
- Produces accurate quotient and remainder.
- Suitable for binary arithmetic in processors.

Limitations

- Extra restoring step increases execution time.
- Slower than the Non-Restoring Division Algorithm.
- Requires more arithmetic operations.

Comparison: Restoring vs Non-Restoring Division

Feature	Restoring Division	Non-Restoring Division
Restoration	Required if result is negative	Not required
Speed	Slower	Faster
Hardware Complexity	Simple	Slightly Complex
Number of Operations	More	Fewer
Efficiency	Moderate	High



4. Accuracy of Calculation (20%)

- Dividend = 1101_2
- Divisor = 0011_2
- Quotient = 0100_2 (4_{10})
- Remainder = 0001_2 (1_{10})
- Verification: $(3 \times 4) + 1 = 13$, confirming the result is correct.

5. Interpretation of Results (15%)

- The Restoring Division Algorithm successfully computes $13 \div 3$ using binary arithmetic.
- Whenever the subtraction produces a negative accumulator, the divisor is added back to restore the previous value.
- The final quotient is 4, and the remainder is 1, which satisfies the division rule.
- Although restoring division is simple and reliable, it requires additional restoring operations, making it slower than the Non-Restoring Division Algorithm.
- Therefore, restoring division is commonly used in educational examples and simple processor designs.

5.A scientific application requires precise handling of real numbers using the IEEE 754 standard for floating-point representation. Given a decimal number (e.g., −13.25), analyze how it is represented in both single precision and double precision formats, including sign, exponent, and mantissa fields. Further, explain how floating-point addition is performed between two such numbers, highlighting issues like normalization, rounding, and precision loss.

1. Problem Understanding (20%)

A processor stores real numbers using the IEEE 754 Single-Precision Floating-Point Standard.

Given Number: −13.625

Requirements:

- Convert the decimal number to binary.
- Normalize the binary number.
- Determine the Sign bit, Exponent, and Mantissa (Fraction).
- Write the final 32-bit IEEE 754 representation.
- Explain why floating-point representation is preferred over fixed-point representation.

2. Method Selection (20%)

The IEEE 754 Single-Precision Standard (32-bit) is used because it provides a standard method to represent real numbers accurately.

IEEE 754 Single Precision Format

Field	Number of Bits
Sign	1 bit
Exponent	8 bits
Mantissa (Fraction)	23 bits

The exponent uses a Bias = 127.

3. Solution Process (25%)

Step 1: Convert Decimal to Binary

Integer Part (13)

13 ÷ 2 = 6 remainder 1

6 ÷ 2 = 3 remainder 0

$3 \div 2 = 1$ remainder 1

$1 \div 2 = 0$ remainder 1

So,

$13_{10} = 1101_2$

Fractional Part (0.625)

Multiply repeatedly by 2:

$0.625 \times 2 = 1.25 \rightarrow 1$

$0.25 \times 2 = 0.5 \rightarrow 0$

$0.5 \times 2 = 1.0 \rightarrow 1$

Therefore,

$0.625_{10} = 0.101_2$

Hence,

$13.625_{10} = 1101.101_2$

Since the number is negative,

$-13.625 = -1101.101_2$

Step 2: Normalize the Binary Number

Move the binary point so that only one digit remains before it.

1101.101_2

$= 1.101101 \times 2^3$

Step 3: Sign Bit

The number is negative.

Sign = 1

Step 4: Exponent

Actual exponent

3

Bias = 127

Exponent = 127 + 3 = 130

Convert 130 into binary

$130_{10} = 10000010_2$

Step 5: Mantissa (Fraction)

Remove the leading 1 from the normalized form.

Normalized number

1.101101

Mantissa

10110100000000000000000

(23 bits)

Step 6: Final IEEE 754 Representation

Field	Value
Sign	1
Exponent	10000010
Mantissa	10110100000000000000000

Final 32-bit Binary

1 10000010 10110100000000000000000

Or,

11000001010110100000000000000000

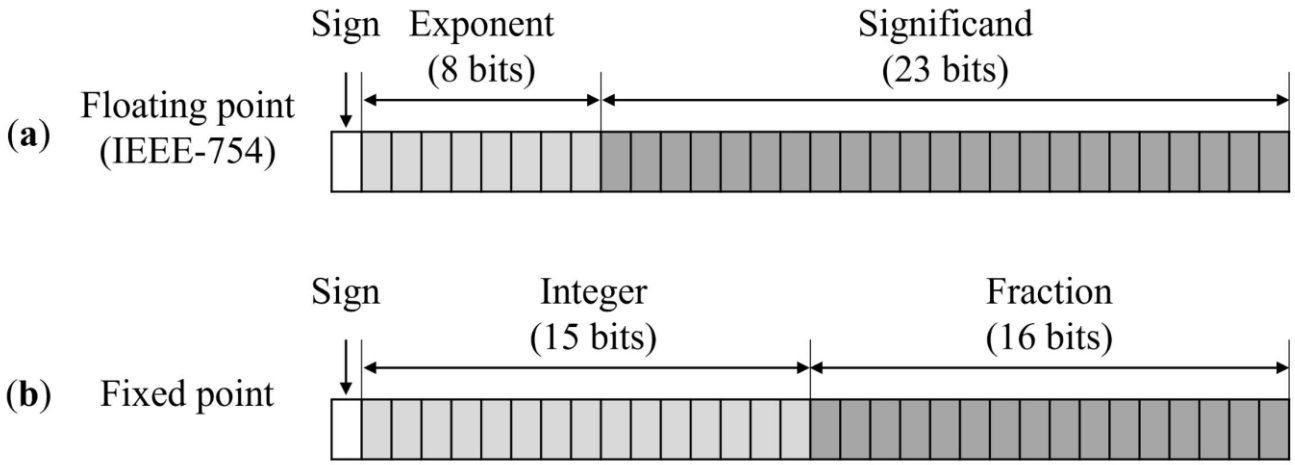
Verification

Component	**Value**
Sign	1
Exponent	130 (10000010_2)
Mantissa	10110100000000000000000
Decimal Number	−13.625

The IEEE 754 representation is correct.

Floating-Point vs Fixed-Point Representation

Feature	Floating-Point	Fixed-Point
Range	Very Large	Limited
Precision	High	Lower
Supports Fractions	Yes	Limited
Scientific Calculations	Excellent	Poor
Memory	More	Less
Applications	AI, Graphics, Scientific Computing	Embedded Systems



Advantages of IEEE 754 Floating-Point

- Represents very large and very small numbers.
- Provides higher precision for real numbers.
- Standard format used across processors.
- Suitable for scientific and engineering calculations.
- Widely used in graphics, machine learning, and numerical computing.

Applications

- Scientific calculations

- Computer graphics
- Artificial Intelligence
- Weather forecasting
- Digital signal processing

4. Accuracy of Calculation (20%)

- Decimal number −13.625 converted correctly to binary.
- Normalized form: 1.101101×2^3.
- Sign bit = 1.
- Exponent = 10000010.
- Mantissa = 10110100000000000000000.
- Final 32-bit IEEE 754 representation:

  11000001010110100000000000000000

5. Interpretation of Results (15%)

- The IEEE 754 standard stores −13.625 using 32 bits divided into a sign bit, exponent, and mantissa.
- The sign bit indicates that the number is negative, the exponent stores the scaled power of two using a bias of 127, and the mantissa stores the significant digits of the normalized binary number.
- This representation provides high precision and a wide numeric range, making it more suitable than fixed-point representation for scientific computations, graphics processing, and modern processors.
- Hence, IEEE 754 floating-point representation is the standard used in most computer systems for handling real numbers.